

Object-oriented Programming in OCaml

Programming Language & Compiler

School of Computer Science, Wuhan University

March 15, 2025

Variants

OCaml 提供关键字 `type` 来定义变体 (Variants):

```
type t = C1 [of t1] | ... | Cn [of tn]
```

`t` 是变体名称, `C1...Cn` 是类型构造器 (constructor), `t1...tn` 是构造器对应的类型。当类型构造器是 `Unit` 类型时, 变体就近似于其他高级语言中的枚举类型。

你可以尝试定义一个日期类型 `days`, 记录一周内的七天:

```
(* days 类型定义了一周的前三天 *)  
type days = Mon of unit | Tue of unit | Wed of unit;;
```

Variants

当类型构造器都是 Unit 类型的时候，OCaml 允许你忽略所有的 of t1, ..., of tn, 此时和其他高级语言的枚举类型非常相似：

```
type days = Mon | Tue | Wed;;  
let x = Mon;;
```

```
> val x : days = Mon
```

变体也可以作为类型别名：

```
(* 在当前上下文中给 int 一个类型别名 *)  
type intalias = int;;
```

Variants

你可以递归地定义变体类型，通过在类型构造器中加入当前正在定义的变体名称实现递归类型，以实现一个链表类型为例：

```
type linked = Empty | Cons of int * linked;;  
let lst = Cons(1, Cons(2, Cons(3, Empty))));;
```

```
> type linked = Empty | Cons of int * linked
```

```
> val lst : linked = Cons (1, Cons (2, Cons (3, Empty)))
```

Variants

变体允许你使用类型变量 'a 给类型构造器添加参数。

可以使用类型变量改进链表类型：

```
type 'a linked = Empty | Cons of 'a * 'a linked;;
```

此时 linked 是一个类型构造器：接收一个类型作为参数，然后返回这个类型组成的链表类型：

```
let lst_str = Cons("0", Cons("Caml", Empty));;
```

```
> val lst_str : string linked = Cons ("0", Cons ("Caml",  
Empty))
```

Variants

上面的案例中类型构造器 `linked` 的参数 `'a` 是隐式确定的。你也可以显式地指定具体类型，就像把 `int` 作为参数传递给 `linked` 那样：

```
(* 通过类型注解指定类型 *)
```

```
let lst : int linked = Cons(1, Cons(2, Cons(3, Empty)));;
```

```
(* 使用类型别名 *)
```

```
type int_linked = int linked;;
```

```
type str_linked = string linked;;
```

Variants

多态变体（Polymorphic Variants）是 OCaml 中另一个强大的特性。通过使用多态变体，你在实现一个具有多态返回类型的函数时，无需提前定义好一个变体。使用反引号 `：

```
let f = function
  | 0 -> `Infinity
  | 1 -> `Finite 1
  | n -> `Finite (-n)
```

```
> val f : int -> [> `Finite of int | `Infinity ] = <fun>
```

在使用这个特点时，多态变体也可以被称为“匿名变体”

Variants

函数也可以使用多态变体作为参数类型，你可以设计一个函数 f 的反函数 g :

```
let g = function
  | `Infnity -> 0
  | `Finite 1 -> 1
  | `Finite n -> -n;;
```

```
val g : [< `Finite of int | `Infnity ] -> int = <fun>
```


Variants

符号 `>` 代表开放类型，除了 `[]` 内已经包含的构造器，还允许其他的构造器。`<` 代表封闭类型，只允许包含 `[]` 内的构造器。

开放类型允许你编写能够处理未完全指定构造器集合的代码，保留多态性，确保函数可以与其他代码组合使用，比如：

```
let combined = `A 32 (* 类型为 [> `A of int] *)  
let extended = `B "OCaml" (* 类型为 [> `B of string] *)  
let lst = [combined; extended]  
  
> val lst : [> `A of int | `B of string ] list = [`A 32; `B  
"OCaml"]
```

Records

记录（record）是一组字段的集合。想要在 OCaml 中使用记录，你首先需要使用 `type` 关键字定义一个记录类型：

```
type record_name = {  
    name: type;  
    name: type;  
}
```

`record_name` 是这种记录类型的名称，`name` 和 `type` 分别指字段的名称和类型。

Records

记录（record）是一组字段的集合。想要在 OCaml 中使用记录，你首先需要使用 `type` 关键字定义一个记录类型：

```
type record_name = {  
    name: type;  
    name: type;  
}
```

`record_name` 是这种记录类型的名称，`name` 和 `type` 分别指字段的名称和类型。

Records

以定义一个宝可梦记录类型为例：

```
type pokemonType = {  
    name: string;  
    hp: int;  
    ptype: string;  
}
```

```
let pokemon = { name = "psyduck"; hp = 50; ptype =  
"water" };;
```

```
val pokemon : pokemonType = {name = "psyduck"; hp = 50;  
ptype = "water"}
```

Mutable

OCaml 提供关键字 `mutable` 标记记录中的某个字段，这会导致该字段可以变化，语法：

```
type record_name = {  
    mutable name: type  
}
```

Mutable

以宝可梦类型为例，为其添加一个可变的 level 字段：

```
type pokemonType = { ... mutable level: int; }
```

再定义一个 1 级的可达鸭：

```
let pokemon = { name = "psyduck"; hp = 50; ptype = "water";  
level = 1 };
```

使用 <- 运算符改变可达鸭的等级为 2 级：

```
pokemon.level <- 2;; pokemon.level;;
```

```
> - : int = 2
```

Refs

OCaml 提供引用（reference）来帮助处理可变数据。

关键字 `ref` 用来定义引用；`!` 用来解引用，取出引用里面的值；`:=` 用来更新引用内包含的值。

```
(* 定义一个引用 *)
```

```
let r = ref ...
```

```
(* 解引用 *)
```

```
let ... = !r
```

```
(* 修改引用值 *)
```

```
r := ...
```

Refs

```
let r = ref 5;;
```

```
let x = !r;;
```

```
r := 10;;
```

```
let y = !r;;
```

```
> val r : int ref = {contents = 5}
```

```
> val x : int = 5
```

```
> - : unit = ()
```

```
> val y : int = 10
```

更新是作为副作用完成的，返回 `unit` 类型。

Refs

在 OCaml 中，引用实际上是具有单个可变字段的记录：

```
type 'a ref = { mutable contents : 'a; }
```

'a ref 是一个记录类型，它具有一个名为 contents 的单个字段，该字段用 mutable 关键字标记。

Exception

定义无参数异常构造器的语法：

```
exception E
```

E 是异常构造器名称。异常构造器也可以携带参数：

```
exception E of t
```

E 是异常构造器名称，of t 指明异常携带了参数及参数对应的类型，t 是类型构造器。

Exception

(* 无参数的异常 *)

```
exception MyError
```

(* 返回错误码的异常 *)

```
exception CodeError of int
```

(* 返回错误信息的异常 *)

```
exception MsgError of string
```

每个异常值都是 `exn` 类型:

```
let msgerror = MsgError "Bad Gateway";;
```

```
> val msgerror : exn = MsgError "Bad Gateway"
```

Exception

抛出异常使用 `raise` 关键字:

```
raise e
```

`e` 是异常值。

OCaml 提供 `try ... with` 结构用于异常处理:

```
try
```

```
  (* 可能抛出异常的表达式 *)
```

```
with
```

```
  | Exception1 -> (* 处理异常1 *)
```

Module

OCaml 规定使用关键字 `module` 定义模块，语法如下：

```
module ModuleName = struct
  module_items
end
```

`ModuleName` 是模块名。`module_items` 是模块内部的项，包括 `let` `type` 等多种定义语句或者嵌套定义语句。

模块名必须以大写开头。最好使用大驼峰命名法命名模块。

Module

以定义一个 Color 模块为例：

```
module Color = struct
  (* 定义颜色类型 *)
  type t = Red | Green | Blue

  (*初始颜色*)
  let red = Red
  let green = Green
  let blue = Blue

  (* 将颜色转换为字符串 *)
```

```
let to_string = function
  | Red -> "Red"
  | Green -> "Green"
  | Blue -> "Blue"
```

(* 按顺序获取下一个颜色 *)

```
let next = function
  | Red -> Green
  | Green -> Blue
  | Blue -> Red
```

```
end
```

```
let red = Color.red
```

```
> module Color :  
  sig  
    type t = Red | Green | Blue  
    val red : t  
    val green : t  
    val blue : t  
    val to_string : t -> string  
    val next : t -> t  
  end  
> val red : Color.t = Color.Red
```


Module

另一个更准确的模块定义的语法描述是：

```
module ModuleName = module_expression
```

struct 表达式仅仅是 module_expression 的一种。

我们可以给予某个 OCaml 模块一个别名，比如：

```
module MyList = List
```

此时，你就可以使用 MyList.<item> 来调用 List 上的项。

Module

模块的定义允许为空的 `struct` 表达式:

```
module Mod = struct end
```

OCaml 同样支持嵌套模块 (Nested Module), 你可以在一个模块里面定义并使用另一个模块:

```
module M = struct
  module Inner = struct let x = 42 end
  let y = Inner.x
end
```

Module

OCaml 使用签名（signature）来标识一个模块的类型。当我们手动指定模块的签名时，签名还可以作为模块的接口（interface），就像 C++ 中的头文件那样，指定模块的哪些组件可以由外部访问，以及暴露哪些类型。模块签名的定义语法如下：

```
module type ModuleType = sig
  specifications
end
```

或者

```
module type ModuleType = module_type_expression
```

Module

为了处理嵌套模块，签名也可以具备嵌套结构。

签名并不需要包含模块中所有的项，一些模块内部的函数可以不在签名中定义，这些函数就不会暴露给外部。

```
module type ModType = sig
  val f : int -> int
  val g : int -> int
end
module Mod : ModType = struct
  let f x = x + 1
  let g x = x * 2
end
```

```
    let h x = x - 1
end
let result = Mod.f 3 + Mod.g 4
(* let result = Mod.h 5 will fail*)
```

Module

为了处理嵌套模块，签名也可以具备嵌套结构。

签名并不需要包含模块中所有的项，一些模块内部的函数可以不在签名中定义，这些函数就不会暴露给外部。

```
module type ModType = sig
  val f : int -> int
  val g : int -> int
end
module Mod : ModType = struct
  let f x = x + 1
  let g x = x * 2
end
```

```
    let h x = x - 1
end
let result = Mod.f 3 + Mod.g 4
(* let result = Mod.h 5 will fail*)
```

在这个例子中，调用模块中的 `f` 和 `g` 是成功的，但是如果想要调用 `h` 就无法通过静态检查：

```
> Error: Unbound value Mod.h
```

Module

在一个模块内定义的所有项的作用域是这个模块内部，想要在外部使用项需要满足：

1. 项已经在签名中被声明
2. 使用 `<模块名>.<项>` 的格式使用

比如：

```
module type SigM = sig
  val x : int
  val y : int
```



```
end;;  
module M : SigM = struct  
  let x = 5  
  let y = 10  
  let z = 15  
end;;
```

```
M.x;;
```

```
> int = 5
```

Module

OCaml 提供了一个关键字 `open`，它可以把跟在 `open` 后面的模块在当前的作用域内“打开”。当调用打开后的模块中的项的时候，不需要再添加 `<模块名>.`，比如：

```
module type SigM = sig
  val x : int
  val y : int
end;;
module M : SigM = struct
  let x = 5
  let y = 10
```

```
let z = 15  
end;;
```

```
open M;; (* 打开模块 M *)
```

```
x;; (* 尝试使用x *)
```

```
> int = 5
```

你还可以使用 `let ... in ...` 把 `open` 限定在更小的作用域中

```
let lower_trim s = let open String in  
  s |> trim |> lowercase_ascii (* open 在 in 后面的区域生效 *)
```

Module

OCaml 提供了模块包含来解决面向对象编程中的继承问题，包含 (includes) 的语法如下：

```
(* 当模块 A 尝试包含模块 B 时，模块 A 的签名应当包含模块 B 的签名 *)  
module type moduleASig = type  
    include moduleBSig  
end  
  
(* 模块 A 包含模块 B *)  
module moduleA : moduleASig = struct  
    include moduleB  
end
```

Module

包含使得一个模块包含所有由另一个模块定义的项，以及一个模块签名包含所有另一个模块签名定义的项。其作用类似于把一个模块（签名）的内部代码在另一个模块（签名）的内部进行“复制粘贴”。

`include` 和 `open` 导致的效果看上去很相似，但是二者截然不同：
`open` 的作用是直接暴露目标模块的项，允许省略模块名前缀，但不改变当前模块的结构，仅影响作用域；而 `include` 则是将目标模块原样复制到当前模块中，类似于代码展开，从而成为当前模块的组成部分。这意味着 `include` 会实际改变模块的结构，并允许通过后续定义覆盖被包含的内容，而 `open` 仅影响编译时的名称解析。

Module

OCaml 中的函子（Functor）是参数化的模块，类似于以模块作为参数和返回值的一个“函数”。从一个简单的案例入手：

```
(* 定义一个非常简单的模块签名 *)
```

```
module type X = sig
```

```
  val x : int
```

```
end
```

```
(* 定义一个函子 IncX *)
```

```
module IncX (M: X) = struct
```

```
  let x = M.x + 1
```

```
end
```

Module

OCaml 中的函子（Functor）是参数化的模块，类似于以模块作为参数和返回值的一个“函数”。从一个简单的案例入手：

```
(* 定义一个非常简单的模块签名 *)
```

```
module type X = sig
```

```
  val x : int
```

```
end
```

```
(* 定义一个函子 IncX *)
```

```
module IncX (M: X) = struct
```

```
  let x = M.x + 1
```

```
end
```

```
> module type X = sig val x : int end  
> module IncX : functor (M : X) -> sig val x : int end
```

OCaml 推断 IncX 是一个函子，接受模块签名为 x 的模块 M 作为参数，返回的模块具有模块签名 sig val x : int end。

Module

函子的定义有两种等价语法:

```
module F (M : S) = struct
  ...
end
```

```
module F = functor (M : S) -> struct
  ...
end
```

Module

和普通函数一样，函子也具备“高阶”的特质，这意味着你可以把多个模块作为参数（或者说每次接受一个模块作为参数，返回一个匿名的函子作为结果，这个匿名函子会继续应用到下一个参数模块）：

```
module F (M1 : S1) ... (Mn : Sn) = struct
  ...
end
module F = functor (M1 : S1) -> ... -> functor (Mn : Sn) ->
struct
  ...
end
```

Module

你可以显式定义函子的类型：

```
module type FunctorType = functor (ParamModule :  
  ParamSignature) -> ResultSignature
```

ParamModule 是参数模块，ParamSignature 和 ResultSignature 都是签名表达式。

```
module type X = sig  
  val x : int  
end  
module type FXY = functor (A : X) -> sig val y : int end
```

```
module A : X = struct
  let x = 1
end
```

```
module F : FXY = functor (A : X) -> struct
  let y = A.x + 1
end
```

```
module B : Y = F(A)
```

```
let yinB = B.y;;
```

```
> val yinB : int = 2
```

Module

OCaml 的 Map 模块提供了一个函子 Make。Make 接受一个特定形式的模块作为输入，返回在给定模块基础上建立的 Map（二叉搜索树），作为参数输入的模块必须满足如下模块签名：

```
module type OrderedType = sig
  (* 键的类型 t *)
  type t
  (* 用于键之间比较大小关系的函数：当大小相同时返回 0，第一个参数小于第二个参数时返回负数，反之正数 *)
  val compare : t -> t -> int
end
```

Module

Make 的返回模块支持对于一组键值对的大部分操作：

```
module type S = sig
  type key
  type 'a t
  val empty: 'a t
  val mem: key -> 'a t -> bool
  val add: key -> 'a -> 'a t -> 'a t
  val find: key -> 'a t -> 'a
  ...
end
```

Module

OCaml 中的 Set 模块用于创建和操作不可变的有序集合，使用平衡二叉树实现。和 Map 模块类似，OCaml 在 Set 模块中也实现了一个函子 Make。参数模块需要满足下面的签名：

```
module type OrderedType = sig
  (* 键的类型 t *)
  type t
  (* 用于键之间比较大小关系的函数：当大小相同时返回 0，第一个参数小于第二个参数时返回负数，反之正数 *)
  val compare : t -> t -> int
end
```

Class

定义一个类的基本语法：

```
class name =  
  object  
    val ...  
    method ...  
  end
```

形象点说，`object ... end` 内定义了一个对象的内容，而关键字 `class` 给这一个对象做了一层“包装”，使一个对象变成一类对象，并且给这一类对象赋予一个名称，还是其具备继承等多种功能。

Class

和很多高级语言一样，在定义类的时候你可以传入参数。此时 `class` 就和函数相似，接受一个参数，返回一个类：

```
class name = fun param ->
  object
    val ...
    method ...
  end
```

(* 等价写法 上面那种写法的语法糖 *)

```
class name param =
  object
```

```
    val ...  
    method ...  
end
```

```
class counter initial =  
  object  
    val zero = 0  
    val mutable x = initial  
    method get = x  
    method add = x <- x + 1  
    method setzero = x <- zero  
  end;;
```

Class

使用关键字 `new` 实例化一个对象，并且使用 `#` 调用类上的方法：

```
(* 创建一个新对象 *)  
let count = new counter 3;;  
(* 调用 add 方法 *)  
count#add;;  
let result = count#get;;  
  
> val count : counter = <obj>  
> - : unit = ()  
> val result : int = 4
```

Class

对象内的方法可以调用该对象内的其他方法，这需要显式的写出

`object(self):`

```
class pokemon =  
  object(self)  
    (* 生命 *)  
    val mutable hp = 100  
    (* 魔力值 *)  
    val mutable mp = 10  
    (* 进行攻击 需要知道还能放什么技能 *)  
    method attack = print_endline ("attack! " ^ self#skill)
```

```
(* 计算能不能放技能 *)
```

```
method skill = match mp with
```

```
  | 0 -> "no mp"
```

```
  | _ -> mp <- mp - 1; "fireball"
```

```
end;;
```

```
let charmander = new pokemon;;
```

```
charmander#attack
```

```
> attack! fireball
```

Class

在某些情况下，你可以不通过类创建一个对象，叫做直接对象 (Immediate Object)。

```
let p =  
  object  
    val mutable x = 0  
    method get_x = x  
    method move d = x <- x + d  
  end;;
```

p 已经是一个对象，不需要再次实例化。直接对象不能被继承。

Class

你可以使用类型变量来使栈支持多种数据类型。比如：

```
class ['a] stack =  
  object (self)  
    val mutable list = ([] : 'a list)      (* instance  
variable *)  
    method push x =                          (* push method *)  
      list <- x :: list  
    method pop =                             (* pop method *)  
      let result = List.hd list in  
      list <- List.tl list;  
      result
```

```
method peek =                                (* peek method *)
  List.hd list
method size =                                (* size method *)
  List.length list
end;;
```

类的名称前的 [...] 绑定了类对应的类型参数。['a] 这个类型参数表明 stack 类是一个泛型类，也叫多态类，是一组对应具体数据类型的类的类（“class of classes”）。

当使用多态的变量时，你必须在类名前加上 `['a]`。这是由 OCaml 的类型系统决定的。实例变量不能直接是未绑定的多态类型，你需要把 `'a` 绑定到类的类型参数上。

如果你在某个类中实现了一个多态方法，即使你没有声明多态的类上变量（比如上文的 `'a list`），你仍然需要在类上绑定类型参数：

```
class ['a] listExec =  
  object  
    method add (x:'a) (l:'a list) = x::l  
  end;;
```

Class

OCaml 支持你定义多个类型参数，比如：

```
class ['a, 'b] pair =  
  object  
    val mutable fst = (None : 'a option)  
    val mutable snd = (None : 'b option)  
    method set x y = fst <- Some x; snd <- Some y  
    method get = (fst, snd)  
  end;;
```

这里 ['a, 'b] 表示 pair 类有两个类型参数，允许存储两种不同类型的值。

Class

你可以使用关键字 `virtual` 来声明一个虚方法（虚变量）。和很多高级语言的虚成员一样，你不必在声明时给出具体实现，具体实现在随后的子类中给出。包含虚方法的类也必须用 `virtual` 来标记，并且不能被实例化，也就是说，不能创建该类的任何对象。

被标记为 `virtual` 的类也可以支持多态，你依然可以使用 `['a', 'b, ...]` 来绑定和使用类型变量。

Class

```
class virtual pokemon =  
  object  
    val virtual name : string  
    val virtual hp : int  
    val virtual atk : int  
  
    method get_name = name  
    method get_hp = hp  
    method virtual attack : pokemon -> int  
  end;;
```

Class

你可以使用 `inherit` 关键字来继承另一个类（称之为父类），并且给出父类上的虚方法的实现。

```
class virtual animal =  
  object  
    (* 虚方法 *)  
    method virtual speak : unit  
  end
```

```
class dog =  
  object
```

```
(* 继承自 animal 类 *)  
inherit animal  
(* 实现父类上的虚方法 *)  
method speak = print_endline "Woof!"  
end
```

如果你想要使用父类上已经实现的属性和方法，当你使用简单的 `inherit` 继承时，父类的方法会被直接合并到子类中，通过显式的定义 `self` 后直接通过 `self#` 调用。如果你想要区分子类和父类，可以在继承时使用关键字 `as` 给父类起一个别名，然后通过该别名来调用。

Class

例如:

```
class base =  
  object  
    (* 定义一个私有变量 *)  
    val mutable counter = 0  
    method get_counter = counter  
    method incr = counter <- counter + 1  
    method print_counter = print_endline (string_of_int  
counter)  
  end
```

```
class derived =  
  object(self)  
    inherit base as super (* 给父类起一个别名 super *)  
    method reset_and_print =  
      (* 调用父类的方法 *)  
      super#incr;  
      super#print_counter;  
      print_endline ("Current counter: " ^ string_of_int  
(super#get_counter))  
    end
```


Class

如果你想要重写父类上已经实现的方法，不需要任何关键字，只需要在子类中直接重新定义：

```
class base =  
    object  
        method greet = print_endline "Hello from base"  
    end  
class derived =  
    object(self)  
        inherit base
```

```
method greet = print_endline "Hello from derived"  
end
```

当你重写父类上的方法后，在不给予父类一个别名的情况下再次使用 `self#` 试图调用父类上的方法时，你只会调用到子类上重写后的方法。只有给予父类一个别名并且通过别名才可以正确调用父类上的方法。

Class

在 OCaml 中，多继承允许一个类同时继承多个父类的行为和属性（称为“多继承”）。实现方式是通过在类定义中连续使用多个 `inherit` 语句。当多个父类重复定义了一个同名方法后，在不使用别名的情况下会保留最后一个继承的类中的实现。例如：

```
class parent1 =  
  object  
    method greet = print_endline "Hello from parent1"  
  end
```

```
class parent2 =
```

```
object
  method greet = print_endline "Hello from parent2"
  method farewell = print_endline "Goodbye from parent2"
end
```

```
class child =
  object(self)
    inherit parent1
    inherit parent2
    method greet_all =
      self#greet;
      self#farewell
  end
```

输出结果是：

Hello from parent2

Goodbye from parent2

Class

类的接口（Interface）类似于模块签名，使用 `class type` 来显式的定义一个类的接口：

(* 定义一个接口，包含两个方法 *)

```
class type my_interface =  
  object  
    method method1 : int -> int  
    method method2 : string -> unit  
  end
```

可以在类定义时通过：指定接口.

Class

接口中声明的方法就是你希望向外暴露的内容。当你实现一个类时，可以实现更多的方法，但如果你在外部只使用接口类型来引用这个类，那么只有接口中声明的方法可见。

Class

使用关键字 `private` 定义私有方法。私有方法不会暴露给外部（不会在类的接口中出现），它们只能被同一对象的其他方法调用。

私有方法是会被子类继承的（默认情况下它们在子类中是可见的），除非它们被接口隐藏。同时，私有方法可以在子类中被转化为公有：只需要重新定义该方法。

Class

```
class my_class =  
  object (self)  
    method public_method =  
      print_endline "This is a public method.";  
      (* 在类内部调用私有方法 *)  
      self#private_method  
  
    method private private_method =  
      print_endline "This is a private method."  
  end
```

```
let () =  
  let obj = new my_class in  
  obj#public_method;  
  (* 以下调用将导致编译错误，因为 private_method 是私有的 *)  
  obj#private_method
```

Class

OCaml 的 `Oo` 模块 是整个对象系统的核心组成部分。

- 克隆对象 `Oo.copy`: 该函数能生成一个对象的浅拷贝

```
class my_class =  
  object (self)  
    method copy = Oo.copy self  
  end
```

- 标识对象 `Oo.id`: 返回一个整数, 这个整数唯一标识当前运行时中一个对象。这对于判断两个对象是否为同一物理实例 (即使它们在结构上看起来一样) 非常有用。

END.